

Angular Avançado · Módulos 3 e 7

Sessão 8

Serviços e DI (cont.): providers e scopes

João Gonçalves · joaogoncalves.net · Making the complicated simple

Onde ficámos na Sessão 7

O Router fechado e o bloco A do capítulo 5

Router (fechado)

- `loadComponent` · preloading
- Query string e fragment
- Path vs hash

DI: os conceitos

- DI = IoC aplicada às dependências
- Token · provider · injector · instância
- `inject()` só num **injection context** (`NG0203`)

No código

- `@Service()` → `FlightClient` no root
- `findResource(from, to)`
- `FlightSearch` → `FlightClient` → `ConfigService`

Por resolver: ir a About e voltar ainda apaga a pesquisa e o cesto. A resposta é uma store — fica para a Sessão 9.

Agenda de hoje

01	**Providers**	Trocar implementações sem mexer em quem pede	≈ 25 min
02	**Extras**	useValue · InjectionToken	≈ 15 min
03	**Scopes**	Root, rota e componente — quanto vive cada instância	≈ 25 min
04	**Onde ficámos**	As stores abrem a Sessão 9	≈ 5 min



Bloco B · 5.1.6 – 5.1.8

Trocar implementações: providers

O token diz o que se pede — o provider diz o que se recebe

O token: a chave do registo

Token	Exemplo	Quando
Classe	FlightClient	token e implementação são a mesma coisa (o normal)
Abstract class	LanguageService	um contrato, várias implementações
InjectionToken<T>	new InjectionToken<string>('API_URL')	valores que não são classes
Interface □	interface LanguageService {}	não serve: desaparece na compilação

```
// TypeScript
interface Lang { get(): string }
abstract class LanguageService {
  abstract getUserLang(): string;
}

// JavaScript (o que corre)
// interface: …nada
class LanguageService {}
```

O `inject()` corre no browser: precisa de um **valor** em runtime para usar como chave. Uma interface não existe lá.

O provider: a receita

Receita	O injector…	Exemplo	Quando
useClass	cria uma instância da classe	{ provide: LanguageService, useClass: BrowserLanguageService }	**hoje**
useValue	entrega o valor tal como está	{ provide: API_URL, useValue: 'https://…' }	configuração · testes
useFactory	chama a função e entrega o resultado	{ provide: LanguageService, useFactory: () => … }	criar depende de outras coisas
useExisting	reutiliza o de outro token	{ provide: OldClient, useExisting: FlightClient }	alias, sem 2.ª instância

`provide` = o que se **pede** · `use…` = o que se **recebe**. Trocar a receita não mexe em quem pede.

Quando é criada a instância?

Registar
guarda a
receita
nada é criado

→

1.º inject
o injector
cria
e guarda em cache

→

2.º inject…
entrega a
mesma
instância

Lazy

- Ninguém pede → nunca é criada
- `@Service()` sem uso → retirada do build (tree-shaking)

Quantas?

- **Uma por injector** que tem a receita
- Onde se regista decide quantas e quanto vivem → bloco C



Da teoria ao código

Agora, vamos escrever o código

- Um contrato (abstract class) e duas implementações
- Registar a receita com { provide, useClass } — e ver o NG0201 sem ela
- Esconder a escolha numa provider function
- Opcional: trocar um valor — useValue e InjectionToken

Ficheiros: language.ts · home.ts · app.config.ts (opcional: config-service.ts · api.ts)

O contrato e duas implementações

```ts
// domains/shared/util-common/language.ts
export abstract class LanguageService {        // o TOKEN
  abstract getUserLang(): string;
}

@Service({ autoProvided: false })
export class DefaultLanguageService implements LanguageService {
  getUserLang(): string { return 'en (default)'; }
}

@Service({ autoProvided: false })
export class BrowserLanguageService implements LanguageService {
  getUserLang(): string { return navigator.language + ' (browser)'; }
}
```

Porquê assim

- `autoProvided: false`: pode ser injetado, mas **não** fica registado sozinho
- **abstract class**, não interface: a interface desaparece na compilação
- `implements`: mesmos membros, sem herança

Registar o provider e consumir o token

```ts
// app.config.ts
providers: [
  // provide: o que se pede · useClass: o que se recebe
  { provide: LanguageService,
    useClass: BrowserLanguageService },
],
```

```ts
// shell/home/home.ts
export class Home {
  private readonly language = inject(LanguageService);
  protected readonly userLang = this.language.getUserLang();
}

<!-- home.html -->
<small>Language: {{ userLang }}</small>
```

Sem provider

- `NG0201: No provider found for LanguageService`
- O token existe; falta a **receita**

Short-hand

- `providers: [FlightClient]`
- = `{ provide: FlightClient, useClass: FlightClient }`
- = `@Service()` na própria classe

Provider functions: esconder os detalhes

```ts
// language.ts
export type LanguageConfig = 'default' | 'browser';

export function provideLanguageService(
  config: LanguageConfig = 'default',
): Provider[] {
  if (config === 'browser') {
    return [{ provide: LanguageService, useClass: BrowserLanguageService }];
  }
  return [{ provide: LanguageService, useClass: DefaultLanguageService }];
}

// app.config.ts
providers: [ /* … */ provideLanguageService('browser') ],
```

O padrão provide…

- Quem configura escolhe uma **opção**, não uma classe
- Já o usamos: `provideRouter(...)`, `provideBrowserGlobalErrorListeners()`
- Arrays aninhados são aplanados

Opcional: trocar o ConfigService com useValue

O mesmo token, outra receita

```
// app.config.ts
providers: [
  // …
  {
    provide: ConfigService,
    useValue: {
      apiUrl: 'http://localhost:3001/flights',   // api:full
    } satisfies ConfigService,
  },
],
```

```
TS2322: Type '"http://localhost:3001/flights"' is not
assignable to type '"http://localhost:3000/flights"'

// config-service.ts — anotar o tipo
readonly apiUrl: string = 'http://localhost:3000/flights';
```

Porquê

- Sem `: string`, o `readonly` fica com o **tipo literal** da string
- `satisfies`: o compilador verifica o objeto (o `useValue` é `any`)
- Funciona porque o TypeScript é **estrutural**
- Ninguém que pede o `ConfigService` mudou
- Outra API: `api:full` (`db-full-500.json`, porta 3001)

Opcional: um valor isolado com InjectionToken

```ts
// data/api.ts
export const API_URL = new InjectionToken<string>('API_URL', {
  factory: () => 'http://localhost:3000/flights',   // valor por omissão
});

// data/flight-client.ts
private readonly apiUrl = inject(API_URL);         // tipo: string
// … url: this.apiUrl

// app.config.ts — só para trocar
{ provide: API_URL, useValue: 'http://localhost:3001/flights' },   // api:full
```

Quando

- **Um valor** → `InjectionToken`
- Vários valores juntos → classe de configuração
- Com `factory`: valor por omissão, sem provider

"Porque não importar a constante?" Com `import`, o valor fica fixo no build. Com um token, basta outro provider.

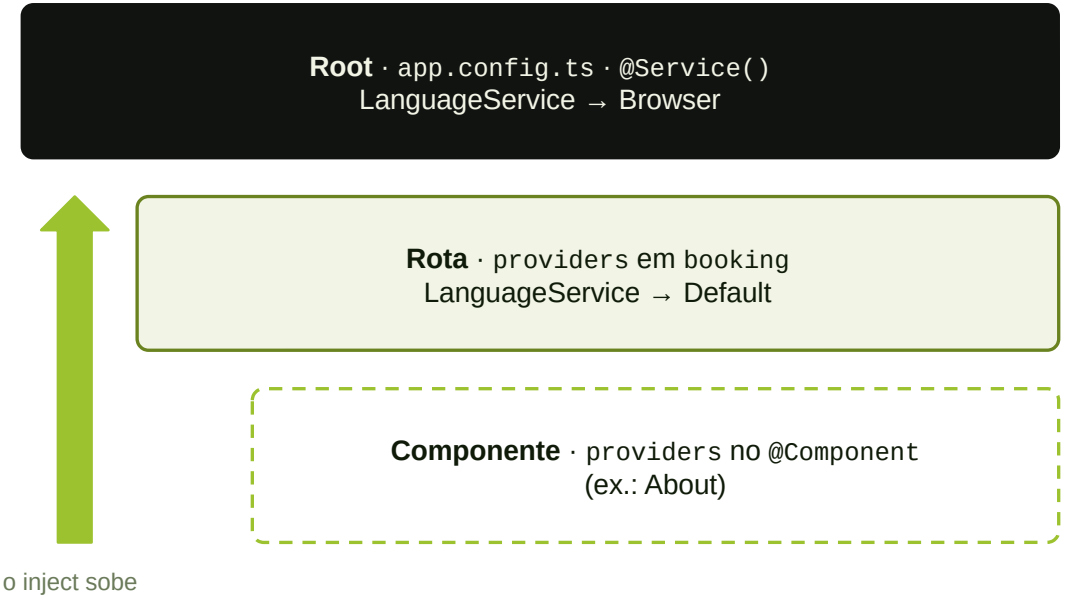


Bloco C · 5.1.9 – 5.1.11

Scopes: onde vive cada instância

Root, rota e componente — e quanto tempo cada instância dura

A hierarquia de injectors



Como se resolve

- Começa no injector **mais próximo** e **sobe**
- O primeiro provider encontrado **ganha** — sombreia os de cima
- Cada injector com provider cria a **sua** instância
- Ninguém o fornece → `NG0201`

Como a **prototype chain** em JavaScript: procura aqui, depois no pai… o mais próximo ganha. No topo, o **NullInjector** só sabe lançar `NG0201`.

Mudar a procura

Para saber que existem — raros em código de aplicação

Opção	O que muda	Exemplo de uso
{ optional: true }	ninguém fornece → null, em vez de NG0201	valor global opcional, com default local
{ self: true }	procura só no próprio injector	garantir uma instância local
{ skipSelf: true }	começa no injector do pai	componente recursivo (árvore de pastas)
{ host: true }	não passa do componente host	par acoplado: lista e itens

```
// About (S9b): o do próprio componente e o do pai
protected readonly parentLang = inject(LanguageService, { skipSelf: true }).getUserLang();
```

Em código existente: `@Optional()`, `@Self()`, `@SkipSelf()`, `@Host()` nos parâmetros do constructor.



Da teoria ao código

Agora, vamos escrever o código

- O mesmo token com receitas diferentes no root, na rota e no componente
- Ver na consola quando cada instância nasce e morre
- Auto cleanup dos injectors das rotas

Ficheiros: about.ts · ticketing.routes.ts · booking-navigation.ts · app.config.ts

Serviços ao nível do componente

```
// shell/about/about.ts
@Component({
  selector: 'app-about',
  templateUrl: './about.html',
  providers: [
    { provide: LanguageService, useClass: DefaultLanguageService },
  ],
})
export default class About {
  protected readonly userLang = inject(LanguageService).getUserLang();
}
```

Página	Mostra	Porquê
Home	pt-PT (browser)	sobe até ao root
About	en (default)	provider do próprio componente

Tempo de vida

- Uma instância **por instância** do componente
- Partilhada com os **filhos**
- Morre com o componente (ver a consola: criado / destruído)
- Para componentes muito acoplados

Serviços ao nível da rota

```ts
// domains/ticketing/ticketing.routes.ts
{
  path: 'booking',
  component: BookingNavigation,
  providers: [
    { provide: LanguageService, useClass: DefaultLanguageService },
  ],
  children: [ /* flight-search, flight-edit/:id, … */ ],
},
```

```ts
// app.config.ts — Angular 21.1+, experimental
provideRouter(routes, /* … */ withExperimentalAutoCleanupInjectors()),
```

Tempo de vida

- Visível na rota **e em todas as child routes**
- Por omissão: **nunca** é destruído
- Com auto cleanup: destruído ao **sair** da rota

Ideal para estado partilhado por uma feature com várias rotas — e que deve desaparecer quando se sai dela.

Onde ficámos

O capítulo 5 continua na Sessão 9

Hoje: providers e scopes

- Token ≠ implementação: `{ provide, useClass }`
- Provider functions: `provideLanguageService('browser')`
- `useValue` · `InjectionToken` com `factory`
- Root → rota → componente: o mais próximo **ganha**
- Auto cleanup dos injectors das rotas

Sessão 9

- **Stores**: estado que sobrevive à navegação
- `FlightStore` no root · `linkedSignal`
- Depois: **guards** funcionais (e interceptors)

Ainda por resolver: ir a About e voltar apaga a pesquisa. Na Sessão 9 o estado passa para uma store no root.